

Análisis Léxico y Tokenización

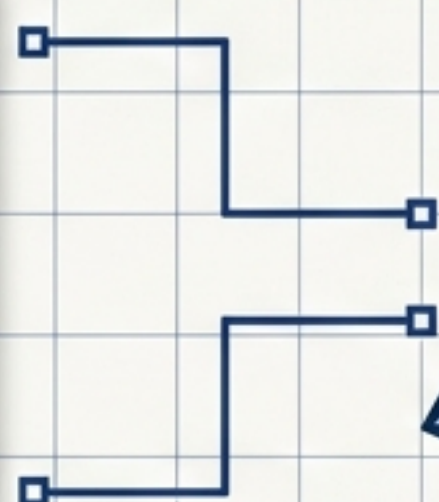
De la teoría de Autómatas a la implementación práctica (Flex & Python)

```
{  
  "universidad": "Universidad Nacional Experimental de Guayana",  
  "proyecto": "Ingeniería en Informática",  
  "unidad": "Lenguajes y Compiladores",  
  "autores": ["Marquez, F.", "Longart, D.", "Martinez, F.", "Ortiz, S.",  
              "Valencia, H."]  
}
```


¿Cómo lee una máquina?

Caos Humano
(Caracteres Crudos)

```
let x=10;fn()  
if{y}let x=10  
;fn()if{y}
```



Orden de la Máquina
(Tokens)

TOKEN_LET

TOKEN_ID(x)

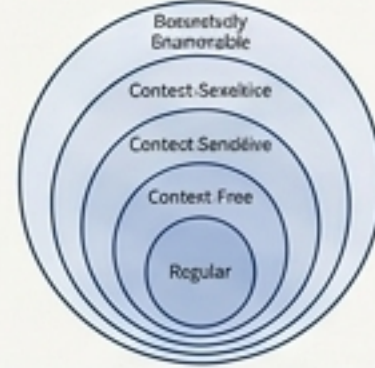
TOKEN_ASIG(=)

TOKEN_NUM(10)

El análisis léxico es la fase inicial del compilador. Actúa como el primer filtro de seguridad, verificando la validez de las estructuras léxicas según reglas formales antes del análisis sintáctico.

Fundamentos Teóricos: El Cerebro del Lexer

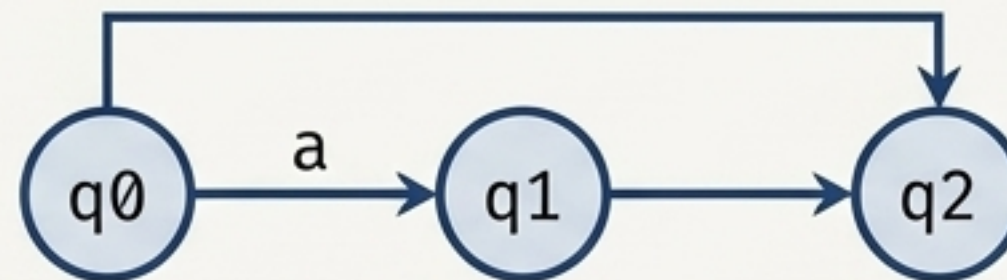
Teoría: Jerarquía de Chomsky



Patrones:
Expresiones Regulares

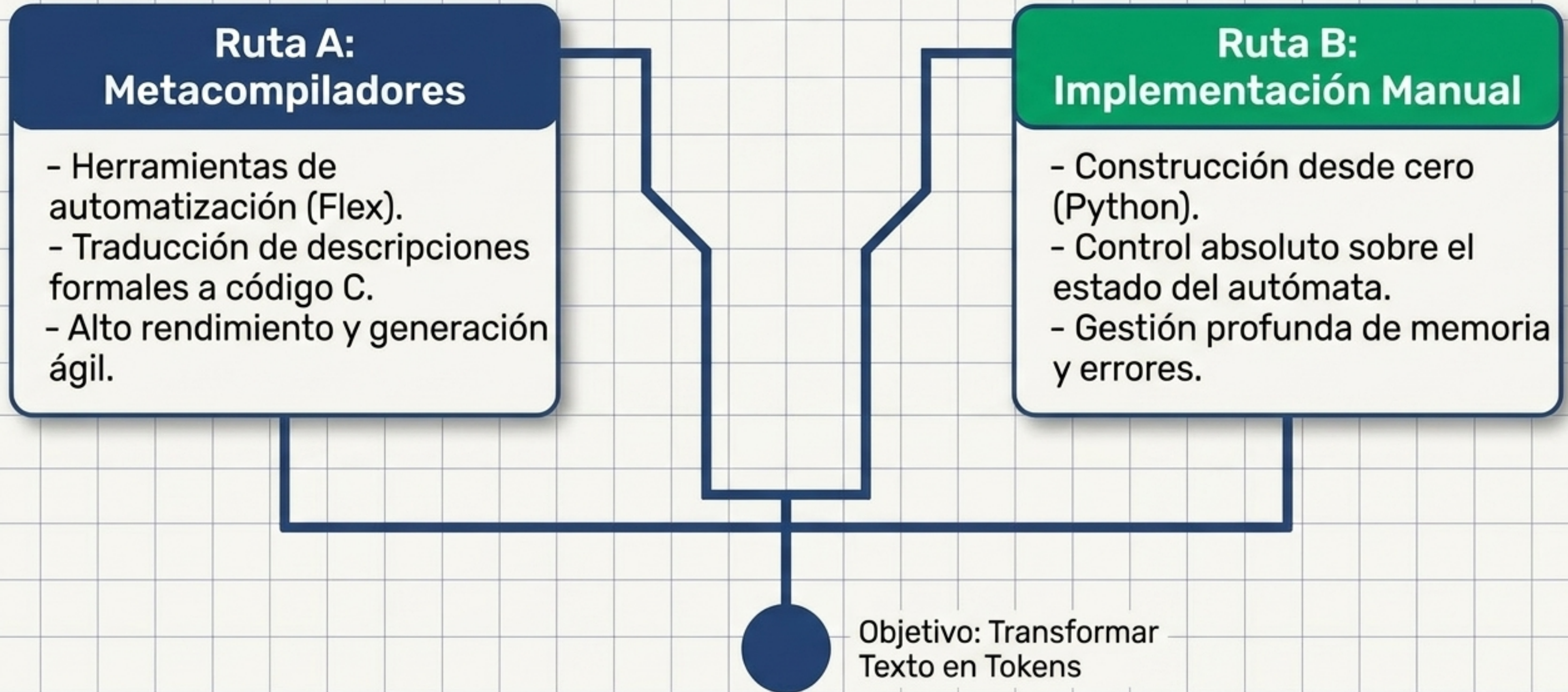
```
[a-zA-Z_][a-zA-Z0-9_]*
```

Máquina:
Autómata Finito
Determinista (AFD)



Los Lexers validan matemáticamente lenguajes regulares escaneando coincidencias carácter por carácter, ejecutando transiciones de estado en un AFD.

La Bifurcación Metodológica

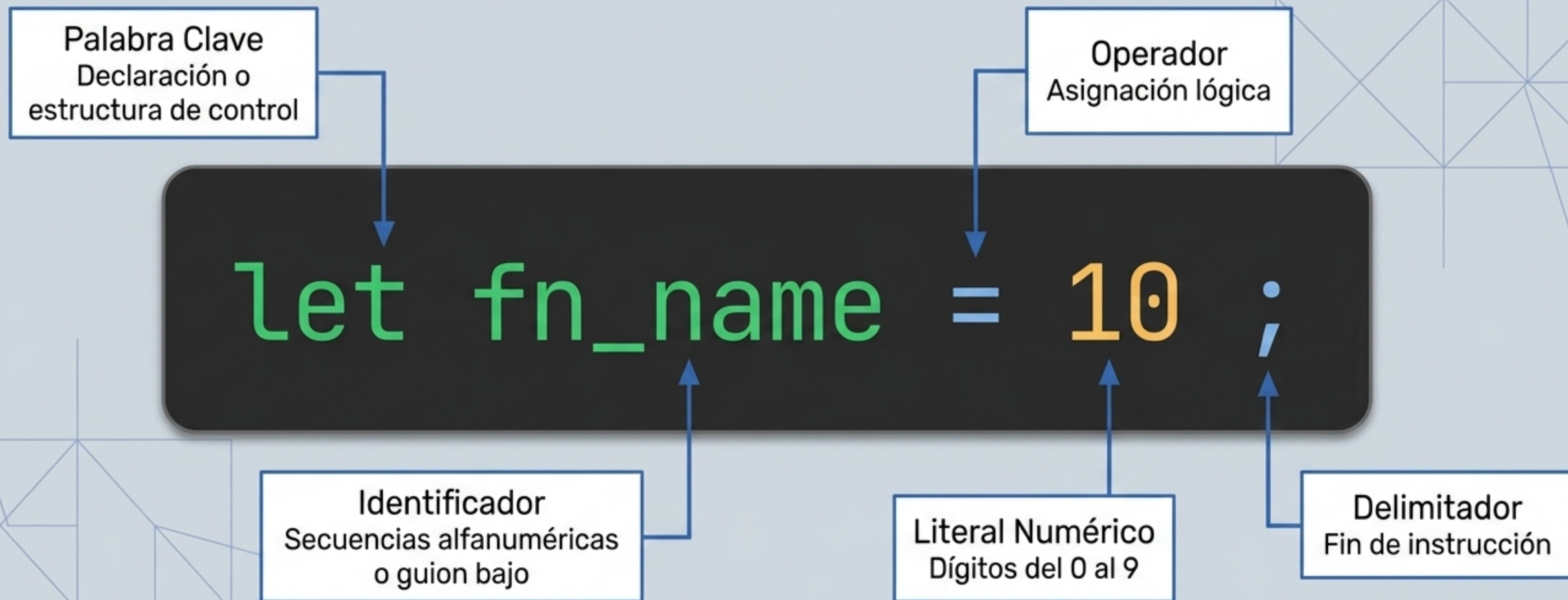


Ruta A: El Metacompilador Flex



Flex no es un compilador convencional que traduce a lenguaje máquina. Es un generador que crea el motor de búsqueda y clasificación en C basándose en las definiciones del programador.

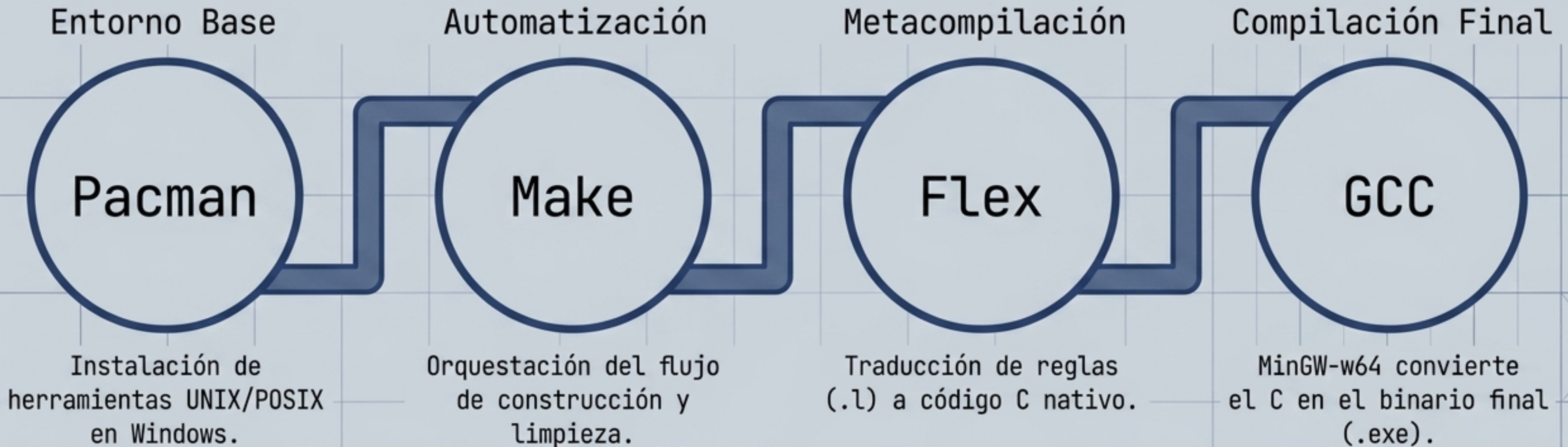
Anatomía del Lenguaje L (Subconjunto de Rust)



Nota: Espacios, tabulaciones y saltos de línea son detectados pero ignorados intencionalmente por el motor.

La Línea de Ensamblaje de Flex

Flujo de trabajo y automatización sobre MSYS2



```
$ make clean
```

Elimina archivos temporales (lex.yy.c y binario) tras la compilación.

Disección del Código Fuente (.l)

1

Opciones:
Finalización sin
dependencias
externas.

```
%{  
#include <stdio.h>  
%}  
%option noyywrap  
%%  
"let" { printf("TOKEN_LET\n"); }  
[0-9]+ { printf("TOKEN_NUMERO(%s)\n", ytext); }  
[a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN_ID(%s)\n", ytext); }  
%%  
int main() { yylex(); return 0; }
```

3

Puntero ytext: Captura
dinámicamente el valor
exacto del carácter
escaneado.

2

Mapeo Directo: La
Expresión Regular a la
izquierda dispara el evento
en C a la derecha.

Ejecución y Resultado de Flex

```
$ echo "let x = 10;" > test.txt
$ ./Lex1 < test.txt
TOKEN_LET
TOKEN_ID(x)
TOKEN_ASIGNACION
TOKEN_NUMERO(10)
TOKEN_PUNTO_COMA
```

Validación Exitosa

El binario C generado por el metacompilador lee la secuencia de caracteres crudos y los clasifica perfectamente según las reglas establecidas en el archivo fuente.

Disección del Código Fuente (.l)

1

Opciones:
Finalización sin
dependencias
externas.

```
%{  
#include <stdio.h>  
%}  
%option noyywrap  
%%  
"let" { printf("TOKEN_LET\n"); }  
[0-9]+ { printf("TOKEN_NUMERO(%s)\n", yytext); }  
[a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN_ID(%s)\n", yytext); }  
%%  
int main() { yylex(); return 0; }
```

3

Puntero yytext: Captura
dinámicamente el valor
exacto del carácter
escaneado.

2

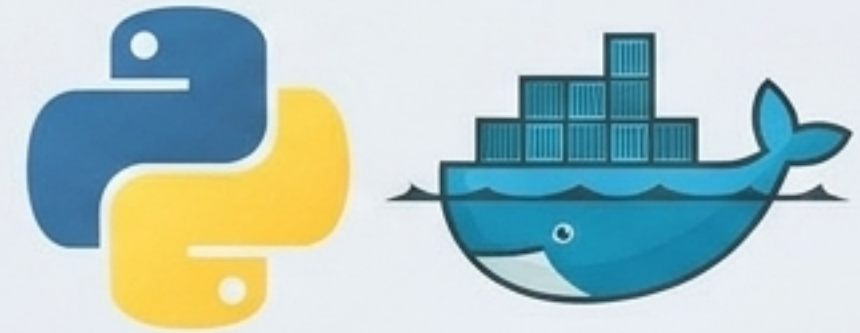
Mapeo Directo: La
Expresión Regular a la
izquierda dispara el evento
en C a la derecha.

```
$ echo "let x = 10;" > test.txt  
$ ./Lex1 < test.txt  
TOKEN_LET  
TOKEN_ID(x)  
TOKEN_ASIGNACION  
TOKEN_NUMERO(10)  
TOKEN_PUNTO_COMA
```

Validación Exitosa

El binario C genera
metacompilador
secuencia de car
crudos y los cla
perfectamente
reglas establ
archivo fuer

Ruta B: Implementación Manual



Un analizador léxico creado desde cero.
Operando mediante el módulo nativo 're' de
Python, diseñado específicamente para
procesar e interpretar archivos de
configuración de infraestructura (Dockerfiles).

- Transición de la maquinaria compilada (C) a
un entorno de scripting ágil.

Arquitectura del Motor Lexer Manual

1. Alfabeto (Tuplas)

Diccionario lógico que empareja nombres con Regex.

```
('INSTRUCTION', r'FROM|RUN')
```

2. Motor Lexer

Compilación usando `re.finditer` y grupos nombrados.

```
(?P<name>...)
```

3. Flujo de Datos

Optimización de memoria procesando token por token sin saturar RAM.

```
yield token
```

4. Manejo de Errores

Rastreo exacto del cursor y detención inmediata ante sintaxis inválida.

```
RuntimeError(MISMATCH)
```


Ejecución y Resultado de Python

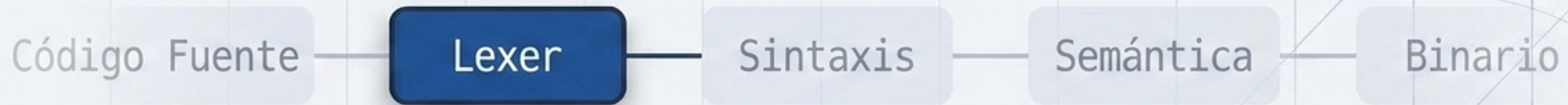
```
> python Lexer_Docker.py  
( 'INSTRUCTION', 'FROM', 3, 4)  
( 'ARGUMENT', 'ubuntu', 3, 9)  
( 'INSTRUCTION', 'RUN', 4, 4)
```



A diferencia de Flex, el control manual del estado en Python permite reportar la posición exacta de cada token (línea y columna), facilitando la depuración posterior.

Síntesis: Flex vs. Python

	Metacompilador (Flex)	Manual (Python)
Eficiencia de Creación	 Alta (Automatizada)	 Media (Programación Manual)
Rendimiento Bruto	 Muy Alto (C Nativo)	 Alto (Módulo interpretado)
Control y Flexibilidad	 Bajo (Caja Negra)	 Extremo (Carácter por carácter)
Manejo de Errores Contextuales	 Básico	 Avanzado (Rastreo exacto de estado)
Consumo de Memoria	 Eficiente (Compilado)	 Eficiente (Patrón Yield)



De Texto a Software

Independientemente de la metodología empleada —C nativo generado por Flex o control granular manual en Python— el análisis léxico es el guardián de la seguridad estructural. Es la frontera donde el lenguaje humano se convierte, finalmente, en el orden lógico de la máquina.

Basado en: The Rust Programming Language (Klabnik & Nichols, 2023), Flex & Bison (Levine, 2009) y Documentación Python 3.12.